PAULT DOCUMENTATION

2026 01 27 Menu Bar Implementation

Source: docs/plans/2026-01-27-menu-bar-implementation.md

> **Theme:** Clean Technical

Generated: February 26, 2026

Menu Bar & Quick Access Implementation Plan

> **Status: COMPLETED** — All phases and tasks implemented successfully.

> **For Claude:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Transform Pault into a menu bar-first app with a popover browser and global hotkey launcher.

Architecture: Add an `AppDelegate` to manage the menu bar status item and NSPopover. Create SwiftUI views for the popover content and hotkey launcher. Use `CGEvent` for global hotkey registration and paste simulation. Keep the existing main window as a secondary management interface.

Tech Stack: SwiftUI, SwiftData, AppKit (NSStatusItem, NSPopover, NSPanel), CGEvent for hotkeys

---

Phase 1: Menu Bar Foundation

Task 1: Add AppDelegate for Menu Bar

Files:

- Create: `Pault/AppDelegate.swift`
- Modify: `Pault/PaultApp.swift`

Step 1: Create AppDelegate

Create `Pault/AppDelegate.swift`:

```
//
// AppDelegate.swift
// Pault
//

import SwiftUI
import AppKit
import SwiftData

class AppDelegate: NSObject, NSApplicationDelegate {
private var statusItem: NSStatusItem?
private var popover: NSPopover?
var modelContainer: ModelContainer?

func applicationDidFinishLaunching(_ notification: Notification) {
setupMenuBar()
}

private func setupMenuBar() {
statusItem = NSStatusBar.system.statusItem(withLength: NSStatusItem.squareLength)

if let button = statusItem?.button {
button.image = NSImage(systemSymbolName: "text.bubble", accessibilityDescription:
"Pault")
button.action = #selector(togglePopover)
button.target = self
}
```

```
popover = NSPopover()
popover?.contentSize = NSSize(width: 320, height: 480)
popover?.behavior = .transient
popover?.animates = true
}

@objc private func togglePopover() {
guard let popover = popover, let button = statusItem?.button else { return }

if popover.isShown {
popover.performClose(nil)
} else {
if let container = modelContainer {
let contentView = MenuBarContentView()
.modelContainer(container)
popover.contentViewController = NSHostingController(rootView: contentView)
}
popover.show(relativeTo: button.bounds, of: button, preferredEdge: .minY)
popover.contentViewController?.view.window?.makeKey()
}
}

func closePopover() {
popover?.performClose(nil)
}
}
```

Step 2: Update PaultApp to use AppDelegate

Replace `Pault/PaultApp.swift`:

```
//
// PaultApp.swift
// Pault
//
// Created by Jonathan Hines Dumitru on 12/16/25.
//

import SwiftUI
import SwiftData

@main
struct PaultApp: App {
@NSApplicationDelegateAdaptor(AppDelegate.self) var appDelegate

var sharedModelContainer: ModelContainer = {
let schema = Schema([
Prompt.self,
Tag.self,
])
let persistentConfig = ModelConfiguration(schema: schema, isStoredInMemoryOnly: false)

do {
return try ModelContainer(for: schema, configurations: [persistentConfig])
} catch {
print("SwiftData ModelContainer load failed: \(error). Falling back to in-memory
store.")
let memoryConfig = ModelConfiguration(schema: schema, isStoredInMemoryOnly: true)
do {
```

```
return try ModelContainer(for: schema, configurations: [memoryConfig])
} catch {
fatalError("Failed to create in-memory ModelContainer fallback: \(error)")
}
}
}()

var body: some Scene {
WindowGroup {
ContentView()
}
.windowResizability(.contentSize)
.defaultSize(width: 900, height: 600)
.modelContainer(sharedModelContainer)
}

init() {
// Pass model container to AppDelegate after initialization
DispatchQueue.main.async { [self] in
appDelegate.modelContainer = sharedModelContainer
}
}
}
```

Step 3: Create placeholder MenuBarContentView

Create `Pault/MenuBarContentView.swift`:

```
//
// MenuBarContentView.swift
// Pault
//

import SwiftUI

struct MenuBarContentView: View {
var body: some View {
VStack {
Text("Pault")
.font(.headline)
Text("Menu bar popover coming soon...")
.foregroundStyle(.secondary)
}
.frame(width: 320, height: 480)
}
}

#Preview {
MenuBarContentView()
}
```

Step 4: Build and verify

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -20`

Expected: BUILD SUCCEEDED. App shows menu bar icon that opens a placeholder popover.

Task 2: Build Menu Bar Popover Layout

Files:

- Modify: `Pault/MenuBarContentView.swift`

Step 1: Create the full popover layout

Replace `Pault/MenuBarContentView.swift`:

```
//
// MenuBarContentView.swift
// Pault
//

import SwiftUI
import SwiftData

enum MenuBarFilter: Hashable {
case favorites
case all
case archived
case tag(Tag)
}

struct MenuBarContentView: View {
@Environment(\.modelContext) private var modelContext
@Query(sort: [SortDescriptor(\Prompt.updatedAt, order: .reverse)]) private var prompts:
[Prompt]

@State private var searchText: String = ""
@State private var selectedFilter: MenuBarFilter = .all
@State private var expandedPromptID: UUID? = nil
@State private var isCreatingNew: Bool = false

private var filteredPrompts: [Prompt] {
var result = prompts

// Apply filter
switch selectedFilter {
case .favorites:
result = result.filter { $0.isFavorite && !$0.isArchived }
case .all:
result = result.filter { !$0.isArchived }
case .archived:
result = result.filter { $0.isArchived }
case .tag(let tag):
result = result.filter { $0.tags?.contains(where: { $0.id == tag.id }) == true &&
!$0.isArchived }
}

// Apply search
if !searchText.isEmpty {
let query = searchText.lowercased()
result = result.filter {
$0.title.lowercased().contains(query) ||
$0.content.lowercased().contains(query) ||
$0.tags?.contains(where: { $0.name.lowercased().contains(query) }) == true
}
}
```

```
return result
}

var body: some View {
VStack(spacing: 0) {
// Search bar
HStack {
Image(systemName: "magnifyingglass")
.foregroundStyle(.secondary)
TextField("Search prompts...", text: $searchText)
.textFieldStyle(.plain)
if !searchText.isEmpty {
Button(action: { searchText = "" }) {
Image(systemName: "xmark.circle.fill")
.foregroundStyle(.secondary)
}
.buttonStyle(.plain)
}
}
.padding(10)
.background(Color(nsColor: .textBackgroundColor))

Divider()

// Filter tabs
HStack(spacing: 0) {
FilterTab(title: "★ Favorites", isSelected: selectedFilter == .favorites) {
selectedFilter = .favorites
}
FilterTab(title: "All", isSelected: selectedFilter == .all) {
selectedFilter = .all
}
FilterTab(title: "Archived", isSelected: selectedFilter == .archived) {
selectedFilter = .archived
}
}
.padding(.horizontal, 8)
.padding(.vertical, 6)

Divider()

// Prompt list
ScrollView {
LazyVStack(spacing: 0) {
ForEach(filteredPrompts) { prompt in
MenuBarPromptRow(
prompt: prompt,
isExpanded: expandedPromptID == prompt.id,
onTap: {
withAnimation(.easeInOut(duration: 0.2)) {
expandedPromptID = expandedPromptID == prompt.id ? nil : prompt.id
}
},
onCopy: { copyToClipboard(prompt.content) },
onPaste: { pastePrompt(prompt.content) },
onToggleFavorite: { toggleFavorite(prompt) },
onArchive: { toggleArchive(prompt) },
```

```
onDelete: { deletePrompt(prompt) }
)
Divider()
}
}
}

Divider()

// Bottom bar
HStack {
Button(action: { isCreatingNew = true }) {
Label("New", systemImage: "plus")
}
.buttonStyle(.plain)

Spacer()

Button(action: openSettings) {
Image(systemName: "gear")
}
.buttonStyle(.plain)
}
.padding(10)
}
.frame(width: 320, height: 480)
.sheet(isPresented: $isCreatingNew) {
NewPromptSheet(isPresented: $isCreatingNew)
}
}

private func copyToClipboard(_ text: String) {
NSPasteboard.general.clearContents()
NSPasteboard.general.setString(text, forType: .string)
}

private func pastePrompt(_ text: String) {
copyToClipboard(text)
// Paste simulation will be added in a later task
}

private func toggleFavorite(_ prompt: Prompt) {
prompt.isFavorite.toggle()
prompt.updatedAt = Date()
try? modelContext.save()
}

private func toggleArchive(_ prompt: Prompt) {
prompt.isArchived.toggle()
prompt.updatedAt = Date()
try? modelContext.save()
}

private func deletePrompt(_ prompt: Prompt) {
modelContext.delete(prompt)
try? modelContext.save()
}
```

```
private func openSettings() {
NSApp.sendAction(Selector(("showSettingsWindow:")), to: nil, from: nil)
}
}

private struct FilterTab: View {
let title: String
let isSelected: Bool
let action: () -> Void

var body: some View {
Button(action: action) {
Text(title)
.font(.caption)
.fontWeight(isSelected ? .semibold : .regular)
.padding(.horizontal, 12)
.padding(.vertical, 6)
.background(isSelected ? Color.accentColor.opacity(0.2) : Color.clear)
.clipShape(RoundedRectangle(cornerRadius: 6))
}
.buttonStyle(.plain)
}
}

private struct MenuBarPromptRow: View {
let prompt: Prompt
let isExpanded: Bool
let onTap: () -> Void
let onCopy: () -> Void
let onPaste: () -> Void
let onToggleFavorite: () -> Void
let onArchive: () -> Void
let onDelete: () -> Void

private var displayTitle: String {
if !prompt.title.isEmpty { return prompt.title }
let preview = prompt.content.prefix(30)
return preview.isEmpty ? "Untitled" : String(preview)
}

var body: some View {
VStack(alignment: .leading, spacing: 0) {
// Header row
Button(action: onTap) {
HStack {
VStack(alignment: .leading, spacing: 2) {
HStack(spacing: 4) {
Text(displayTitle)
.font(.system(size: 13, weight: .medium))
.lineLimit(1)
if prompt.isFavorite {
Image(systemName: "star.fill")
.font(.caption2)
.foregroundStyle(.yellow)
}
}
if let tags = prompt.tags, !tags.isEmpty {
HStack(spacing: 4) {
```

```
ForEach(tags.prefix(2)) { tag in
Text("#\(tag.name)")
.font(.caption2)
.foregroundStyle(.secondary)
}
if tags.count > 2 {
Text("+\(tags.count - 2)")
.font(.caption2)
.foregroundStyle(.tertiary)
}
}
}
}
Spacer()
Image(systemName: isExpanded ? "chevron.up" : "chevron.down")
.font(.caption)
.foregroundStyle(.tertiary)
}
.padding(.horizontal, 12)
.padding(.vertical, 8)
.contentShape(Rectangle())
}
.buttonStyle(.plain)

// Expanded content
if isExpanded {
VStack(alignment: .leading, spacing: 8) {
Text(prompt.content)
.font(.system(size: 12))
.foregroundStyle(.secondary)
.lineLimit(6)
.padding(.horizontal, 12)

HStack(spacing: 8) {
Button(action: onCopy) {
Label("Copy", systemImage: "doc.on.doc")
.font(.caption)
}
.buttonStyle(.bordered)

Button(action: onPaste) {
Label("Paste", systemImage: "doc.on.clipboard")
.font(.caption)
}
.buttonStyle(.bordered)

Spacer()

Menu {
Button(action: onToggleFavorite) {
Label(prompt.isFavorite ? "Unfavorite" : "Favorite", systemImage: "star")
}
Button(action: onArchive) {
Label(prompt.isArchived ? "Unarchive" : "Archive", systemImage: "archivebox")
}
Divider()
Button(role: .destructive, action: onDelete) {
Label("Delete", systemImage: "trash")
```

```
}
} label: {
Image(systemName: "ellipsis.circle")
.font(.caption)
}
.menuStyle(.borderlessButton)
}
.padding(.horizontal, 12)
.padding(.bottom, 8)
}
.background(Color(nsColor: .controlBackgroundColor).opacity(0.5))
}
}
.background(isExpanded ? Color(nsColor: .controlBackgroundColor).opacity(0.3) :
Color.clear)
}
}

private struct NewPromptSheet: View {
@Environment(\.modelContext) private var modelContext
@Binding var isPresented: Bool

@State private var title: String = ""
@State private var content: String = ""

var body: some View {
VStack(spacing: 12) {
Text("New Prompt")
.font(.headline)

TextField("Title", text: $title)
.textFieldStyle(.roundedBorder)

TextEditor(text: $content)
.frame(height: 120)
.border(Color.secondary.opacity(0.3))

HStack {
Button("Cancel") {
isPresented = false
}
.keyboardShortcut(.cancelAction)

Spacer()

Button("Create") {
let prompt = Prompt(title: title, content: content)
modelContext.insert(prompt)
try? modelContext.save()
isPresented = false
}
.keyboardShortcut(.defaultAction)
.disabled(title.isEmpty && content.isEmpty)
}
}
.padding()
.frame(width: 280)
}
```

```
}

#Preview {
MenuBarContentView()
.modelContainer(for: [Prompt.self, Tag.self], inMemory: true)
}
```

Step 2: Build and verify

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -20`

Expected: BUILD SUCCEEDED. Menu bar popover shows full browser with search, filters, and prompt list.

Phase 2: Global Hotkey Launcher

Task 3: Create HotkeyLauncher Window

Files:

- Create: `Pault/HotkeyLauncherWindow.swift`
- Create: `Pault/HotkeyLauncherView.swift`

Step 1: Create the launcher window controller

Create `Pault/HotkeyLauncherWindow.swift`:

```
//
// HotkeyLauncherWindow.swift
// Pault
//

import SwiftUI
import AppKit
import SwiftData

class HotkeyLauncherWindowController {
private var window: NSPanel?
private var modelContainer: ModelContainer?

init(modelContainer: ModelContainer?) {
self.modelContainer = modelContainer
}

func show() {
if window == nil {
createWindow()
}

guard let window = window else { return }

window.center()
window.makeKeyAndOrderFront(nil)
NSApp.activate(ignoringOtherApps: true)
}

func hide() {
```

```
window?.orderOut(nil)
}

func toggle() {
if window?.isVisible == true {
hide()
} else {
show()
}
}

private func createWindow() {
let panel = NSPanel(
contentRect: NSRect(x: 0, y: 0, width: 500, height: 320),
styleMask: [.nonactivatingPanel, .titled, .fullSizeContentView],
backing: .buffered,
defer: false
)
panel.isFloatingPanel = true
panel.level = .floating
panel.titleVisibility = .hidden
panel.titlebarAppearsTransparent = true
panel.isMovableByWindowBackground = true
panel.backgroundColor = .clear
panel.isOpaque = false
panel.hasShadow = true

if let container = modelContainer {
let contentView = HotkeyLauncherView(onDismiss: { [weak self] in
self?.hide()
})
.modelContainer(container)

panel.contentView = NSHostingView(rootView: contentView)
}

self.window = panel
}
}
```

Step 2: Create the launcher view

Create `Pault/HotkeyLauncherView.swift`:

```
//
// HotkeyLauncherView.swift
// Pault
//

import SwiftUI
import SwiftData

struct HotkeyLauncherView: View {
@Environment(\.modelContext) private var modelContext
@Query(sort: [SortDescriptor(\Prompt.updatedAt, order: .reverse)]) private var prompts:
[Prompt]

@State private var searchText: String = ""
@State private var selectedIndex: Int = 0
```

```
@State private var showingActions: Bool = false
@State private var selectedPrompt: Prompt? = nil

let onDismiss: () -> Void

private var filteredPrompts: [Prompt] {
let nonArchived = prompts.filter { !$0.isArchived }

if searchText.isEmpty {
// Show favorites first, then recent
let favorites = nonArchived.filter { $0.isFavorite }
let others = nonArchived.filter { !$0.isFavorite }
return Array((favorites + others).prefix(9))
}

let query = searchText.lowercased()
return nonArchived.filter {
$0.title.lowercased().contains(query) ||
$0.content.lowercased().contains(query) ||
$0.tags?.contains(where: { $0.name.lowercased().contains(query) }) == true
}
}

var body: some View {
VStack(spacing: 0) {
if showingActions, let prompt = selectedPrompt {
// Action view
actionView(for: prompt)
} else {
// Search view
searchView
}
}
.frame(width: 500)
.background(VisualEffectView(material: .hudWindow, blendingMode: .behindWindow))
.clipShape(RoundedRectangle(cornerRadius: 12))
.shadow(color: .black.opacity(0.3), radius: 20, y: 10)
.onKeyPress(.escape) {
if showingActions {
showingActions = false
selectedPrompt = nil
return .handled
}
onDismiss()
return .handled
}
.onKeyPress(.downArrow) {
if selectedIndex < filteredPrompts.count - 1 {
selectedIndex += 1
}
return .handled
}
.onKeyPress(.upArrow) {
if selectedIndex > 0 {
selectedIndex -= 1
}
return .handled
}
```

```
.onKeyPress(.return) {
if !showingActions, filteredPrompts.indices.contains(selectedIndex) {
selectedPrompt = filteredPrompts[selectedIndex]
showingActions = true
}
return .handled
}
}

private var searchView: some View {
VStack(spacing: 0) {
// Search field
HStack(spacing: 12) {
Image(systemName: "magnifyingglass")
.font(.title2)
.foregroundStyle(.secondary)

TextField("Search prompts...", text: $searchText)
.font(.title3)
.textFieldStyle(.plain)
.onSubmit {
if filteredPrompts.indices.contains(selectedIndex) {
selectedPrompt = filteredPrompts[selectedIndex]
showingActions = true
}
}
}
.padding(16)

Divider()

// Results
ScrollView {
LazyVStack(spacing: 0) {
ForEach(Array(filteredPrompts.enumerated()), id: \.element.id) { index, prompt in
LauncherResultRow(
prompt: prompt,
index: index,
isSelected: index == selectedIndex,
onSelect: {
selectedPrompt = prompt
showingActions = true
},
onQuickCopy: {
copyToClipboard(prompt.content)
onDismiss()
}
)
.onHover { hovering in
if hovering { selectedIndex = index }
}
}
}
}
.frame(maxHeight: 250)
}
.onChange(of: searchText) { _, _ in
selectedIndex = 0
```

```
}
}

private func actionView(for prompt: Prompt) -> some View {
VStack(spacing: 16) {
Text(prompt.title.isEmpty ? "Untitled" : prompt.title)
.font(.headline)
.lineLimit(1)

HStack(spacing: 12) {
ActionButton(title: "Copy", shortcut: "⌘C", icon: "doc.on.doc") {
copyToClipboard(prompt.content)
onDismiss()
}

ActionButton(title: "Paste", shortcut: "⌘V", icon: "doc.on.clipboard") {
copyToClipboard(prompt.content)
pasteToFrontApp()
onDismiss()
}

ActionButton(title: "Edit", shortcut: "⌘E", icon: "pencil") {
// Open in menu bar popover - to be implemented
onDismiss()
}

ActionButton(title: "Open", shortcut: "⌘O", icon: "arrow.up.forward.app") {
// Open in main app - to be implemented
onDismiss()
}
}
.padding(.horizontal)
}
.padding(20)
.onKeyPress(KeyEquivalent("c"), modifiers: .command) {
copyToClipboard(prompt.content)
onDismiss()
return .handled
}
.onKeyPress(KeyEquivalent("v"), modifiers: .command) {
copyToClipboard(prompt.content)
pasteToFrontApp()
onDismiss()
return .handled
}
}

private func copyToClipboard(_ text: String) {
NSPasteboard.general.clearContents()
NSPasteboard.general.setString(text, forType: .string)
}

private func pasteToFrontApp() {
// Paste simulation - will be implemented in Task 4
DispatchQueue.main.asyncAfter(deadline: .now() + 0.1) {
let source = CGEventSource(stateID: .hidSystemState)
let keyDown = CGEvent(keyboardEventSource: source, virtualKey: 0x09, keyDown: true) // V
key
```

```
let keyUp = CGEvent(keyboardEventSource: source, virtualKey: 0x09, keyDown: false)
keyDown?.flags = .maskCommand
keyUp?.flags = .maskCommand
keyDown?.post(tap: .cghidEventTap)
keyUp?.post(tap: .cghidEventTap)
}
}
}

private struct LauncherResultRow: View {
let prompt: Prompt
let index: Int
let isSelected: Bool
let onSelect: () -> Void
let onQuickCopy: () -> Void

private var displayTitle: String {
if !prompt.title.isEmpty { return prompt.title }
return String(prompt.content.prefix(40))
}

var body: some View {
Button(action: onSelect) {
HStack {
if index < 9 {
Text("■\(index + 1)")
.font(.caption)
.foregroundStyle(.tertiary)
.frame(width: 30)
} else {
Spacer().frame(width: 30)
}

VStack(alignment: .leading, spacing: 2) {
HStack(spacing: 4) {
Text(displayTitle)
.lineLimit(1)
if prompt.isFavorite {
Image(systemName: "star.fill")
.font(.caption2)
.foregroundStyle(.yellow)
}
}
if let tags = prompt.tags, !tags.isEmpty {
HStack(spacing: 4) {
ForEach(tags.prefix(3)) { tag in
Text("#\(tag.name)")
.font(.caption2)
.foregroundStyle(.secondary)
}
}
}
}

Spacer()
}
.padding(.horizontal, 12)
.padding(.vertical, 8)
```

```
.background(isSelected ? Color.accentColor.opacity(0.2) : Color.clear)
.contentShape(Rectangle())
}
.buttonStyle(.plain)
.onKeyPress(KeyEquivalent(Character("\(index + 1)")), modifiers: .command) {
if index < 9 {
onQuickCopy()
return .handled
}
return .ignored
}
}
}

private struct ActionButton: View {
let title: String
let shortcut: String
let icon: String
let action: () -> Void

var body: some View {
Button(action: action) {
VStack(spacing: 6) {
Image(systemName: icon)
.font(.title2)
Text(title)
.font(.caption)
Text(shortcut)
.font(.caption2)
.foregroundStyle(.secondary)
}
.frame(width: 70, height: 70)
.background(Color.secondary.opacity(0.1))
.clipShape(RoundedRectangle(cornerRadius: 8))
}
.buttonStyle(.plain)
}
}

struct VisualEffectView: NSViewRepresentable {
let material: NSVisualEffectView.Material
let blendingMode: NSVisualEffectView.BlendingMode

func makeNSView(context: Context) -> NSVisualEffectView {
let view = NSVisualEffectView()
view.material = material
view.blendingMode = blendingMode
view.state = .active
return view
}

func updateNSView(_ nsView: NSVisualEffectView, context: Context) {
nsView.material = material
nsView.blendingMode = blendingMode
}
}

#Preview {
```

```
HotkeyLauncherView(onDismiss: {})
.modelContainer(for: [Prompt.self, Tag.self], inMemory: true)
}
```

Step 3: Build and verify

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -20`

Expected: BUILD SUCCEEDED

Task 4: Add Global Hotkey Registration

Files:

- Create: `Pault/GlobalHotkeyManager.swift`
- Modify: `Pault/AppDelegate.swift`

Step 1: Create GlobalHotkeyManager

Create `Pault/GlobalHotkeyManager.swift`:

```
//
// GlobalHotkeyManager.swift
// Pault
//

import Foundation
import Carbon
import AppKit

class GlobalHotkeyManager {
static let shared = GlobalHotkeyManager()

private var eventHandler: EventHandlerRef?
private var hotkeyID = EventHotKeyID()
private var hotkeyRef: EventHotKeyRef?
private var callback: (() -> Void)?

private init() {}

func register(keyCode: UInt32, modifiers: UInt32, callback: @escaping () -> Void) {
self.callback = callback

// Unregister any existing hotkey
unregister()

// Set up hotkey ID
hotkeyID.signature = OSType(fourCharCodeFrom: "PALT")
hotkeyID.id = 1

// Register the hotkey
var eventType = EventTypeSpec(eventClass: OSType(kEventClassKeyboard), eventKind:
UInt32(kEventHotKeyPressed))

let status = InstallEventHandler(
GetApplicationEventTarget(),
```

```
{ (_, event, userData) -> OSStatus in
guard let userData = userData else { return OSStatus(eventNotHandledErr) }
let manager = Unmanaged<GlobalHotkeyManager>.fromOpaque(userData).takeUnretainedValue()
manager.callback?()
return noErr
},
1,
&eventType,
Unmanaged.passUnretained(self).toOpaque(),
&eventHandler
)

if status != noErr {
print("Failed to install event handler: \(status)")
return
}

let registerStatus = RegisterEventHotKey(
keyCode,
modifiers,
hotkeyID,
GetApplicationEventTarget(),
0,
&hotkeyRef
)

if registerStatus != noErr {
print("Failed to register hotkey: \(registerStatus)")
}
}

func unregister() {
if let hotkeyRef = hotkeyRef {
UnregisterEventHotKey(hotkeyRef)
self.hotkeyRef = nil
}
if let eventHandler = eventHandler {
RemoveEventHandler(eventHandler)
self.eventHandler = nil
}
}

private func fourCharCodeFrom(_ string: String) -> FourCharCode {
var result: FourCharCode = 0
for char in string.utf8.prefix(4) {
result = (result << 8) + FourCharCode(char)
}
return result
}

deinit {
unregister()
}
}

// Key codes and modifier masks
extension GlobalHotkeyManager {
// Common key codes
```

```
static let keyCodeP: UInt32 = 0x23

// Modifier masks (Carbon)
static let cmdKey: UInt32 = UInt32(cmdKey)
static let shiftKey: UInt32 = UInt32(shiftKey)
static let optionKey: UInt32 = UInt32(optionKey)
static let controlKey: UInt32 = UInt32(controlKey)
}
```

Step 2: Update AppDelegate to use hotkey and launcher

Replace `Pault/AppDelegate.swift`:

```
//
// AppDelegate.swift
// Pault
//

import SwiftUI
import AppKit
import SwiftData
import Carbon

class AppDelegate: NSObject, NSApplicationDelegate {
private var statusItem: NSStatusItem?
private var popover: NSPopover?
private var launcherController: HotkeyLauncherWindowController?
var modelContainer: ModelContainer?

func applicationDidFinishLaunching(_ notification: Notification) {
setupMenuBar()
setupGlobalHotkey()
}

private func setupMenuBar() {
statusItem = NSStatusBar.system.statusItem(withLength: NSStatusItem.squareLength)

if let button = statusItem?.button {
button.image = NSImage(systemSymbolName: "text.bubble", accessibilityDescription:
"Pault")
button.action = #selector(togglePopover)
button.target = self
}

popover = NSPopover()
popover?.contentSize = NSSize(width: 320, height: 480)
popover?.behavior = .transient
popover?.animates = true
}

private func setupGlobalHotkey() {
// Register ■+Shift+P
let modifiers = UInt32(cmdKey | shiftKey)
let keyCode: UInt32 = 0x23 // P key

GlobalHotkeyManager.shared.register(keyCode: keyCode, modifiers: modifiers) { [weak
self] in
self?.toggleLauncher()
}
```

```
}

@objc private func togglePopover() {
guard let popover = popover, let button = statusItem?.button else { return }

if popover.isShown {
popover.performClose(nil)
} else {
if let container = modelContainer {
let contentView = MenuBarContentView()
.modelContainer(container)
popover.contentViewController = NSHostingController(rootView: contentView)
}
popover.show(relativeTo: button.bounds, of: button, preferredEdge: .minY)
popover.contentViewController?.view.window?.makeKey()
}
}

private func toggleLauncher() {
if launcherController == nil {
launcherController = HotkeyLauncherWindowController(modelContainer: modelContainer)
}
launcherController?.toggle()
}

func closePopover() {
popover?.performClose(nil)
}
}
```

Step 3: Build and verify

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -20`

Expected: BUILD SUCCEEDED

Phase 3: App Lifecycle & Preferences

Task 5: Configure App as Menu Bar Agent

Files:

- Modify: `Pault/Info.plist` (or add to project)

Step 1: Create/update Info.plist entries

Add to the app's Info.plist (via Xcode target settings or plist file):

```
<key>LSUIElement</key>
<true/>
```

This can be done in Xcode: Target → Info → Custom macOS Application Target Properties → Add "Application is agent (UIElement)" = YES

Step 2: Update PaultApp to handle window lifecycle

Modify `Pault/PaultApp.swift`:

```
//
// PaultApp.swift
// Pault
//
// Created by Jonathan Hines Dumitru on 12/16/25.
//

import SwiftUI
import SwiftData

@main
struct PaultApp: App {
@NSApplicationDelegateAdaptor(AppDelegate.self) var appDelegate

var sharedModelContainer: ModelContainer = {
let schema = Schema([
Prompt.self,
Tag.self,
])
let persistentConfig = ModelConfiguration(schema: schema, isStoredInMemoryOnly: false)

do {
return try ModelContainer(for: schema, configurations: [persistentConfig])
} catch {
print("SwiftData ModelContainer load failed: \(error). Falling back to in-memory
store.")
let memoryConfig = ModelConfiguration(schema: schema, isStoredInMemoryOnly: true)
do {
return try ModelContainer(for: schema, configurations: [memoryConfig])
} catch {
fatalError("Failed to create in-memory ModelContainer fallback: \(error)")
}
}
}()

var body: some Scene {
WindowGroup {
ContentView()
}
.windowResizability(.contentSize)
.defaultSize(width: 900, height: 600)
.modelContainer(sharedModelContainer)
.commands {
CommandGroup(replacing: .newItem) {
Button("New Prompt") {
// Will be handled by ContentView
}
.keyboardShortcut("n", modifiers: .command)
}
}

Settings {
PreferencesView()
.modelContainer(sharedModelContainer)
}
}
```

```
init() {
DispatchQueue.main.async { [self] in
appDelegate.modelContainer = sharedModelContainer
}
}
}
```

Step 3: Create PreferencesView

Create `Pault/PreferencesView.swift`:

```
//
// PreferencesView.swift
// Pault
//

import SwiftUI
import ServiceManagement

struct PreferencesView: View {
@AppStorage("globalHotkey") private var globalHotkey: String = "⌘⌘P"
@AppStorage("launchAtLogin") private var launchAtLogin: Bool = false
@AppStorage("showDockIcon") private var showDockIcon: Bool = false
@AppStorage("defaultAction") private var defaultAction: String = "showOptions"
@AppStorage("pasteDelay") private var pasteDelay: Double = 100

var body: some View {
TabView {
generalTab
.tabItem {
Label("General", systemImage: "gear")
}

hotkeyTab
.tabItem {
Label("Hotkey", systemImage: "keyboard")
}
}
.frame(width: 450, height: 250)
}

private var generalTab: some View {
Form {
Toggle("Launch at login", isOn: $launchAtLogin)
.onChange(of: launchAtLogin) { _, newValue in
setLaunchAtLogin(newValue)
}

Toggle("Show dock icon", isOn: $showDockIcon)
.onChange(of: showDockIcon) { _, newValue in
setDockIconVisibility(newValue)
}

Picker("Default action", selection: $defaultAction) {
Text("Show options").tag("showOptions")
Text("Copy to clipboard").tag("copy")
Text("Paste to app").tag("paste")
}
```

```
HStack {
Text("Paste delay")
Slider(value: $pasteDelay, in: 0...500, step: 50)
Text("\(Int(pasteDelay))ms")
.monospacedDigit()
.frame(width: 50)
}
}
.padding()
}

private var hotkeyTab: some View {
Form {
HStack {
Text("Global hotkey")
Spacer()
Text(globalHotkey)
.padding(.horizontal, 12)
.padding(.vertical, 6)
.background(Color.secondary.opacity(0.2))
.clipShape(RoundedRectangle(cornerRadius: 6))
// Note: Full hotkey recording would require additional implementation
}

Text("Press ■■P from anywhere to open the quick launcher.")
.font(.caption)
.foregroundStyle(.secondary)
}
.padding()
}

private func setLaunchAtLogin(_ enabled: Bool) {
do {
if enabled {
try SMAppService.mainApp.register()
} else {
try SMAppService.mainApp.unregister()
}
} catch {
print("Failed to set launch at login: \(error)")
}
}

private func setDockIconVisibility(_ visible: Bool) {
if visible {
NSApp.setActivationPolicy(.regular)
} else {
NSApp.setActivationPolicy(.accessory)
}
}
}

#Preview {
PreferencesView()
}
```

Step 4: Build and verify

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -20`

Expected: BUILD SUCCEEDED

Task 6: Add lastUsedAt to Prompt Model

Files:

- Modify: `Pault/Prompt.swift`

Step 1: Update Prompt model

Replace `Pault/Prompt.swift`:

```
//
// Prompt.swift
// Pault
//
// Created by Jonathan Hines Dumitru on 12/16/25.
//

import Foundation
import SwiftData

@Model
final class Prompt {
var id: UUID
var title: String
var content: String
var isFavorite: Bool
var isArchived: Bool
@Relationship var tags: [Tag]?
var createdAt: Date
var updatedAt: Date
var lastUsedAt: Date?

init(id: UUID = UUID(), title: String, content: String, isFavorite: Bool = false,
isArchived: Bool = false, createdAt: Date = Date(), updatedAt: Date = Date(),
lastUsedAt: Date? = nil, tags: [Tag]? = nil) {
self.id = id
self.title = title
self.content = content
self.isFavorite = isFavorite
self.isArchived = isArchived
self.createdAt = createdAt
self.updatedAt = updatedAt
self.lastUsedAt = lastUsedAt
self.tags = tags
}

func markAsUsed() {
self.lastUsedAt = Date()
}
}
```

Step 2: Update launcher to track usage

In `HotkeyLauncherView.swift`, update `copyToClipboard` to mark prompt as used:

Add to the `HotkeyLauncherView` struct:

```
private func copyToClipboard(_ text: String, prompt: Prompt? = nil) {
NSPasteboard.general.clearContents()
NSPasteboard.general.setString(text, forType: .string)
prompt?.markAsUsed()
try? modelContext.save()
}
```

Step 3: Build and verify

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -20`

Expected: BUILD SUCCEEDED

Phase 4: Final Integration

Task 7: Wire Everything Together

Files:

- Modify: `Pault/AppDelegate.swift`
- Modify: `Pault/MenuBarContentView.swift`

Step 1: Add "Open Main Window" to menu bar

Update `MenuBarContentView.swift` - replace the `openSettings` function and add an open main window option:

In the bottom bar section, update to:

```
// Bottom bar
HStack {
Button(action: { isCreatingNew = true }) {
Label("New", systemImage: "plus")
}
.buttonStyle(.plain)

Spacer()

Button(action: openMainWindow) {
Image(systemName: "macwindow")
}
.buttonStyle(.plain)
.help("Open Main Window")

Button(action: openSettings) {
Image(systemName: "gear")
}
.buttonStyle(.plain)
.help("Settings")
}
.padding(10)
```

Add the function:

```
private func openMainWindow() {
NSApp.activate(ignoringOtherApps: true)
if let window = NSApp.windows.first(where: { $0.title.contains("Pault") ||
$0.contentView is NSHostingView<ContentView> }) {
window.makeKeyAndOrderFront(nil)
} else {
// Open new window if none exists
NSApp.sendAction(#selector(NSApplication.newWindowForTab(_:)), to: nil, from: nil)
}
}
```

Step 2: Build final app

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -20`

Expected: BUILD SUCCEEDED

Step 3: Test the complete flow

1. Launch app - should appear in menu bar only (no dock icon)

2. Click menu bar icon - popover with full browser appears

3. Press Cmd++Shift+P - launcher window appears centered on screen

4. Search and select a prompt - action options appear

5. Copy/Paste actions work

6. Settings accessible from menu bar popover

Summary

| Phase | Tasks | Description |

|-------|-------|-------------|

| 1 | 1-2 | Menu bar foundation with popover browser |

| 2 | 3-4 | Global hotkey launcher with fuzzy search |

| 3 | 5-6 | App lifecycle (menu bar agent) and preferences |

| 4 | 7 | Final integration and testing |

Total: 7 tasks

New files created:

- `Pault/AppDelegate.swift`
- `Pault/MenuBarContentView.swift`
- `Pault/HotkeyLauncherWindow.swift`
- `Pault/HotkeyLauncherView.swift`
- `Pault/GlobalHotkeyManager.swift`
- `Pault/PreferencesView.swift`

Modified files:

- `Pault/PaultApp.swift`
- `Pault/Prompt.swift`